

Queue Tracker — Low Level Design (LLD)

Detailed Component Specification, APIs, Data Flow & Implementation

Contents: 1. Component Tree | 2. Types | 3. REST APIs | 4. Socket Events | 5. Backend | 6. Data Flow | 7. State Management | 8. Security | 9. Utilities | 10. Scripts

Project: Queue Tracker v4.0.0 | **Author:** Piximperfect-SK

1. Frontend Component Tree

```
main.tsx
  AuthProvider (JWT-based account auth)
  App.tsx
    Login Screen (name+key OR account login)
      LoginPanel -- JWT login form
      RegisterPanel -- Account registration
    Main App
      Router
        NavigationLogger -- logs route changes
      AppFrame
        AuthProvider + RoleProvider
        Navbar -- nav + presence indicators
        Routes:
          /      RosterPage    -- Shift scheduling
          /tracker TrackerPage -- Live productivity
          /settings SettingsPage -- Handler management
          /logs   LogMonitorPage -- Event monitor
          /admin   AdminPage    -- Role management
        Components: Navbar, ConfirmModal, TrendsSplineChart
        Utils: socket.ts, logger.ts, types.ts
```

2. Type Definitions (types.ts)

```

export interface LogEntry {
  timestamp: string; user: string;
  action: string; details: string;
  type?: 'positive' | 'negative' | 'neutral';
}
export type StandardShiftType = '6AM-3PM' | '1PM-10PM' | '2PM-11PM'
  | '10PM-7AM' | '12PM-9PM' | 'WO' | 'ML' | 'PL' | 'EL' | 'UL' | 'CO' | 'MID-LEAVE';
export type ShiftType = StandardShiftType | string;
export interface Handler { id: string; name: string; isQH: boolean; }
export interface RosterEntry { handlerId: string; date: string; shift: ShiftType; }
export interface DailyStats { handlerId: string; date: string;
  incidents: number; sctasks: number; calls: number; comments: string; }

```

3. REST API Endpoints

Authentication (routes/auth.js)

Method	Endpoint	Rate	Body / Response
POST	/api/register	3/hr	{ username, fullName, password, registrationSecret }
POST	/api/login	3/5min	{ username, password } -> httpOnly JWT cookie
POST	/api/logout	None	Clears token cookie
GET	/api/me	None	Returns user from JWT cookie
GET	/api/csrf-token	None	CSRF token (cookie + JSON)

Role Management (routes/roles.js)

Method	Endpoint	Auth	Notes
GET	/api/roles	Admin	List all users with roles
PUT	/api/roles	Admin	{ userId, role } - prevents last admin demotion

User Lookup (routes/userLookup.js)

4. Socket.IO Events

Client to Server

Event	Payload	Description
join	{ username, accessKey }	Join session; validates access key; triggers presence + state sync
get_initial_data	none	Request full state (handlers, roster, stats)
get_presence	none	Request online users list

update_handlers	Handler[]	Update handlers list (also legacy update_agents)
update_roster	RosterEntry[]	Update roster entries
update_stats	DailyStats[]	Update daily stats
add_log	LogEntry	Add new audit log entry

Server to Client

Event	Payload	Description
init	{ handlers, roster, stats, logs }	Full state on connect/join
presence_updated	string[]	Updated online user list
handlers_updated	Handler[]	Handler list changed
roster_updated	RosterEntry[]	Roster changed
stats_updated	DailyStats[]	Stats changed
log_added	{ dateStr, logEntry }	New log broadcast
error_message	string	Error notification

5. Backend Structure

```

backend/
  server.js           -- Entry: Express + Socket.IO + MongoDB
  package.json        -- ESM, dependencies
  middleware/
    auth.js           -- JWT: optionalAuth, requireAuth, requireRole
    requireRole.js    -- Role-checking middleware factory
  models/
    User.js           -- Mongoose User schema
  routes/
    auth.js           -- /api/login, register, logout, me
    auth_routes.js    -- Alternative auth (modified rate limits)
    roles.js          -- /api/roles GET/PUT (admin only)
    userLookup.js     -- /api/get-role-by-name
  scripts/
    seedAdmin.js      -- Create initial admin user
    testAuth.js       -- Auth smoke tests
    loginAs.js / checkUser.js -- Testing utilities
    rotateAdminPass.js -- Admin password rotation
    generateReport.js  -- PDF report via Puppeteer
    startBackend.ps1  -- PowerShell launcher
  reports/            -- Generated reports

```

6. Data Flow Details

Roster Update (Drag & Drop)

1. @dnd-kit fires onDragEnd with handler ID + drop zone
2. RosterPage determines new shift, calls executeShiftUpdate()
3. Local state + localStorage updated
4. syncData.updateRoster() emits via Socket.IO
5. Server updates MongoDB: State.updateOne({key:'global'}, {\$set: {roster}})
6. Server broadcasts roster_updated to other clients
7. Other clients update local state + localStorage

Name-Based Login

1. User enters name + access key at Access Gate
2. Socket connects; emits 'join' event
3. Server validates access key (if configured)

7. State Management

State	Component(s)	Persistence	Sync
handlers	RosterPage, TrackerPage, SettingsPage	localStorage('handlers')	Socket.IO
roster	RosterPage, TrackerPage	localStorage('roster')	Socket.IO
stats	TrackerPage	localStorage('stats')	Socket.IO
logs	LogMonitorPage	localStorage('logs_YYYY-MM-DD')	Socket.IO + REST
currentUser	App.tsx	localStorage('currentUser')	Local only
accessKey	App.tsx	localStorage('teamAccessKey')	Local only
onlineUsers	App.tsx	None	Socket.IO
user (JWT)	AuthContext	httpOnly cookie	REST /api/me
role	RoleContext	localStorage('userRole')	REST API

8. Security Implementation

Input Sanitizer

```
const sanitize = (text) => {  
  if (typeof text !== 'string') return text;
```

```
return text.replace(/<[>]*>/g, '').trim();
};
```

Applied to: handler names, log details, usernames, access keys (XSS prevention).

Rate Limiting

Limiter	Window	Max
Global HTTP	15 min	100
Login	5 min	3
Registration	1 hour	3
Socket Events	1 sec	20/socket

Role Access Matrix

Feature	Associate	QH	Admin
Roster (/)	✓	✓	✓
Tracker (/tracker)	✓	✓	✓
Settings (/settings)	—	✓	✓
Log Monitor (/logs)	✓	✓	✓
Admin (/admin)	—	—	✓
GET/PUT /api/roles	—	—	✓

9. Utility Functions

socket.ts

```
export const socket = io(SOCKET_URL, {
  transports: ['websocket', 'polling'],
  reconnection: true, reconnectionAttempts: 10,
  withCredentials: true,
});
export const syncData = {
  join, updateHandlers, updateRoster, updateStats, addLog
};
```

logger.ts Functions

Function	Description
----------	-------------

<code>addLog(action, details, type)</code>	Create log entry, emit via socket, store in localStorage
<code>addLogForDate(dateStr, ...)</code>	Create log entry for a specific date
<code>getLogsForDate(dateStr)</code>	Retrieve logs for a date from localStorage
<code>downloadLogsForDate(dateStr)</code>	Open backend download URL for a date
<code>downloadAllLogs()</code>	Open backend download URL for all logs
<code>saveLogsFromServer(allLogs)</code>	Sync server logs into localStorage
<code>saveSingleLogFromServer(...)</code>	Save incoming log with deduplication

10. Scripts & Environment Variables

Available Commands

Command	Description
<code>npm run dev-frontend</code>	Vite dev server (port 5173)
<code>npm run dev-backend</code>	Backend with <code>--watch</code> (port 3001)
<code>npm run build</code>	Build frontend for production
<code>npm run start</code>	Start backend in production

Key Environment Variables

Variable	Default	Description
<code>MONGODB_URI</code>	<code>mongodb://localhost:27017/queue_tracker</code>	MongoDB connection
<code>JWT_SECRET</code>	<code>dev_jwt_secret</code>	JWT signing (min 32 chars in prod)
<code>FRONTEND_URL</code>	<code>http://localhost:5173</code>	CORS origin
<code>TEAM_ACCESS_KEY</code>	None	Shared secret for name-based auth
<code>REGISTRATION_SECRET</code>	None	Required for account registration
<code>PORT</code>	3001	Backend port
<code>VITE_BACKEND_URL</code>	<code>http://localhost:3001</code>	Frontend backend URL

Deployment Topology

```
User Browser
  |-- HTTPS --> Netlify (Frontend SPA)
  |
  |           |-- VITE_BACKEND_URL --> Render (Backend)
```

```
|  
|-- MongoDB Atlas  
|-- WebSocket --> Render (Socket.IO)  
Port 3001
```

Queue Tracker v4.0.0 — Low Level Design Document — Confidential

4. Server registers socket in onlineUsers Map
5. Broadcasts presence_updated to all clients
6. Sends init event with full state to joining client
7. RoleContext refreshes via /api/get-role-by-name

JWT Account Login

1. AuthContext.login() fetches CSRF token from GET /api/csrf-token
2. POSTs to /api/login with username, password, CSRF header
3. Server validates via bcrypt.compare(), generates JWT
4. Sets httpOnly cookie 'token' (8h, sameSite varies by env)
5. Returns user object { username, fullName, role }
6. RoleContext calls /api/me for latest role

Method	Endpoint	Notes
GET	/api/get-role-by-name?name=X	Bridges socket auth with JWT roles

Log Downloads & Health

Method	Endpoint	Notes
GET	/download-logs/:date	Download logs for a date as .txt
GET	/download-all-logs	Complete audit history as .txt
GET	/	"Queue Tracker API is Live!"
GET	/health	"Backend is running"